

国科大作业考试资料-人工智能原理与算法-2024新编-第四次作业整理



八数码与八皇后问题求解

声明：本专题几乎每次作业都有两份答案作为参考，

后附两次考试原题+备期末考指南。

仅做参考交流使用。

生成大量的八数码问题和八皇后问题并用以下算法分别求解：（对于八数码问题，可以自定义合适

的目标状态）

1. 爬山法（最陡上升和首选爬山法）

2. 随机重启爬山法

3. 模拟退火算法

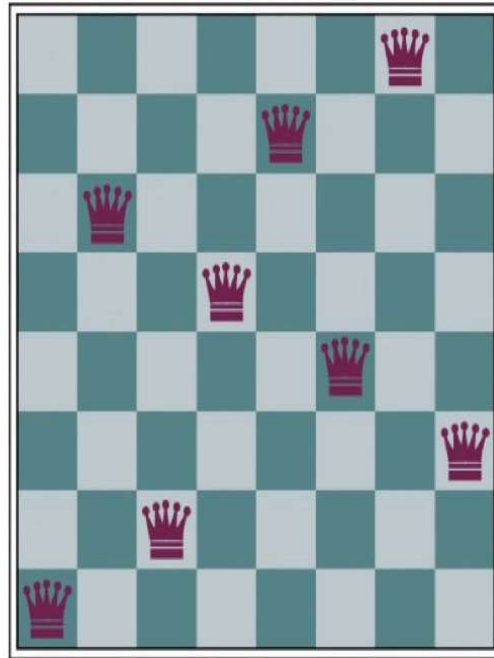
计算搜索耗散和问题的解决率，并用图对比它们和最优解代价的曲线。对结果进行评估。

参考：

下图为八数码问题示意图：



八皇后问题

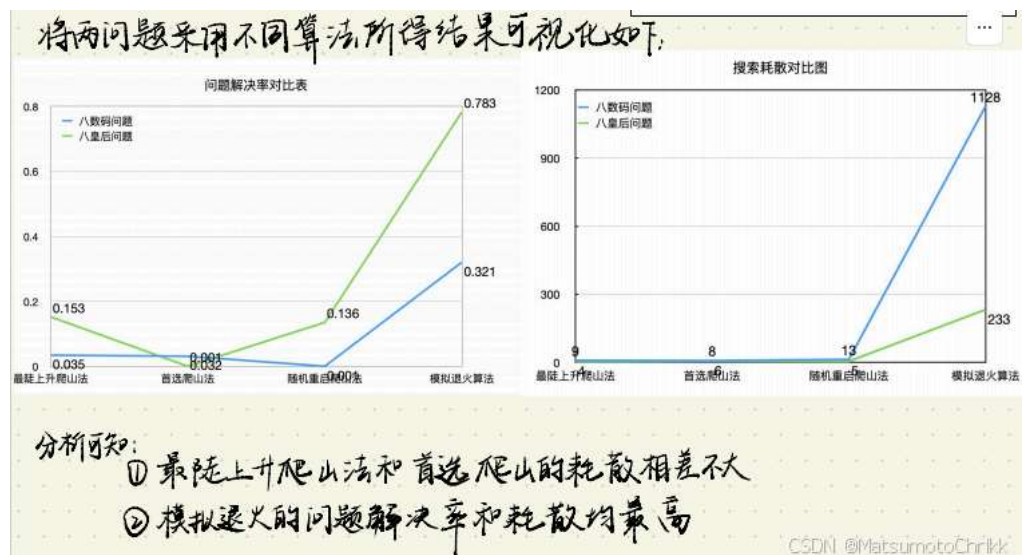


(a)

18	12	14
14	16	13
14	12	18
15	14	14
17	14	17
17	16	16
18	14	17
14	14	13

图 4-3 (a) 8 皇后问题：在棋盘上放置 8 个皇后，使得它们不能互相攻击或对角线上的任何棋子。) 当前状态非常接近于一个解，除了第 4 列和第 7 攻击。(b) 一个 8 皇后状态，其启发式代价估计值 $h = 17$ 。棋盘显示了通过一个可能后继的 h 值。有 8 个移动并列最优，其 $h = 12$ 。爬山法将选择它们

解：



11 最陡上升爬山法

① 八数码问题

```
// 爬山法
bool hillClimbing() {
    for (int trial = 0; trial < 200; trial++) {
        int curManha = Manhattan(); // 当前状态
        int minMan = 99999, minX = 0, minY = 0;
        for (int i = 0; i < 4; i++) { // 在后继状态中找最小值
            int x = row_0 + direction[i][0];
            int y = col_0 + direction[i][1];
            if (x >= 0 && x < 3 && y >= 0 && y < 3) { // 方向可行
                swap(current[row_0][col_0], current[x][y]); // 交换a和相邻位置
                int nextManha = Manhattan();
                if (nextManha < minMan) { // 获取下一状态的最小值
                    minMan = nextManha;
                    minX = x, minY = y;
                }
                swap(current[x][y], current[row_0][col_0]); // 复原a和相邻位置
            }
        }
        if (curManha > minMan) { // 最小值优于当前状态
            swap(current[row_0][col_0], current[minX][minY]);
            row_0 = minX, col_0 = minY;
        }
        if (check()) { // 成功找到解
            totalTrial += trial;
            return true;
        }
    }
    return false;
}
```

```
// 最陡上升爬山法
int steepestAscent() {
    int count = 0;
    for (int i = 0; i < 1000; i++) {
        initial();
        if (hillClimbing())
            count++;
    }
    return count;
}
```

结果:

最陡上升爬山法结果如下
问题解决率: 35/1000
搜索耗散: 9

CSDN @MatsumotoChirikk

② 八皇后问题

```
// 爬山法
bool hillClimbing() {
    // 尝试次数大于1000则判定为无解
    int i, j, k;
    for (int trial = 0; trial <= 100; trial++) {
        // 将初始棋盘数据复制
        for (i = 0; i < 8; i++) {
            for (j = 0; j < 8; j++) {
                temp[i][j] = queens[i][j];
            }
        }
        int minH = 9999, minX = 0, minY = 0, curState;
        for (i = 0; i < 8; i++) {
            for (j = 0; j < 8; j++) {
                for (k = 0; k < 8; k++) {
                    temp[i][k] = 0; // 在计算h(i, j)之前, 对i行所有位置赋值为0
                    h[i][j] = findCollision(i, j); // 寻找h(i, j)
                    if (queens[i][j] == 1) {
                        curState = h[i][j]; // 当前状态的h值
                    }
                    // 先找出冲突个数最小的位置
                    if (h[i][j] < minH) {
                        minH = h[i][j];
                        minX = i;
                        minY = j;
                    }
                }
                for (k = 0; k < 8; k++) {
                    temp[i][k] = queens[i][k]; // 计算h(i, j)之后复原棋盘数据, 避免计算错误
                }
            }
        }
        // 将皇后放在该行冲突最少的位置处
        if (curState > minH) {
            for (int i = 0; i < 8; i++) {
                queens[minX][i] = 0;
                queens[minX][minY] = 1;
            }
            // 判断是否找到解, 有解则返回值为真
            if (check(h)) {
                totalTrial += trial;
                return true;
            }
        }
    }
    return false;
}
```

```
// 最陡上升爬山法
int steepestAscent() {
    int count = 0;
    for (int i = 0; i < 1000; i++) {
        initial();
        if (hillClimbing())
            count++;
    }
    return count;
}
```

最陡上升爬山法结果如下
问题解决率: 153/1000
搜索耗散: 4

CSDN @MatsumotoChirikk

12 首选爬山法

① 八数码问题

```
// 首选爬山法
bool firstchase() {
    for (int trial = 0; trial < 500; trial++) {
        // 随机选取一个优于当前状态的下一步
        bool next = false;
        int times = 0;
        while (!next) {
            int dir = rand() % 4;
            int curManha = Manhattan();
            int x = row_0 + direction[dir][0];
            int y = col_0 + direction[dir][1];
            if (x >= 0 && x < 3 && y >= 0 && y < 3) { // 方向可行
                swap(current[row_0][col_0], current[x][y]);
                int nextManha = Manhattan();
                if (nextManha < curManha) {
                    row_0 = x, col_0 = y;
                    next = true;
                }
                swap(current[x][y], current[row_0][col_0]);
            }
        }
        if (++times > 20) break;
    }
    if (check()) { // 成功找到解
        totalTrial += trial;
        return true;
    }
    return false;
}
```

首选爬山法结果如下
问题解决率: 32/1000
搜索耗散: 8

CSDN @MatsumotoChirikk

② 八皇后问题

```
// 爬山法
bool FirstClimbing() {
    int i, j, k;
    for (int trial = 0; trial <= 100; trial++) { // 尝试次数大于100则判定为无解
        for (i = 0; i < 8; i++) {
            for (j = 0; j < 8; j++) {
                temp[i][j] = queens[i][j]; // 将当前初始状态数据复制到temp
            }
        }
        int h[8][8], curState;
        for (i = 0; i < 8; i++) {
            for (j = 0; j < 8; j++) {
                for (k = 0; k < 8; k++) {
                    temp[i][k] = 0; // 在计算h(i, j)之前, 对i行所有位置赋值都为0
                    h[i][j] = findCollision(i, j); // 查找h(i, j)
                    if (queens[i][j] == 1) {
                        curState = h[i][j]; // 当前状态的值
                    }
                    for (k = 0; k < 8; k++) {
                        temp[i][k] = queens[i][k]; // 计算h(i, j)之后恢复原数据, 避免计算错误
                    }
                }
            }
        }
        bool better = false; // 随机选择一个优于当前状态的下一状态
        int next, nextState, times = 0;
        while (!better) {
            next = rand() % N;
            nextState = h[next][next];
            if (nextState < curState) {
                better = true;
            }
            if (++times > 100) break;
        }
        if (better) {
            for (int i = 0; i < 8; i++) {
                queens[next][i] = 0;
                queens[i][next] = 1; // 放置皇后
            }
            if (check(h)) {
                totalTrial += trial; // 判断是否找到解, 有解则返回值为真
                return true;
            }
        }
        return false;
    }
}
```

首选爬山法结果如下
问题解决率: 136/1000
搜索耗散: 6

CSDN @MatsumotoChirikk

2. 随机重启爬山法 ① 八数码问题

```
// 爬山法
bool hillClimbing() {
    for (int trial = 0; trial < 200; trial++) {
        int curManha = Manhattan(); // 当前状态
        int minMan = 99999, minX = 0, minY = 0;
        for (int i = 0; i < 4; i++) { // 在随机状态中找最小值
            int x = row_0 + direction[i][0];
            int y = col_0 + direction[i][1];
            if (x >= 0 && x < 3 && y >= 0 && y < 3) { // 方向可行
                swap(current[row_0][col_0], current[x][y]); // 交换0和相邻位置
                int nextManha = Manhattan();
                if (nextManha < minMan) { // 获取下一状态的最小值
                    minMan = nextManha;
                    minX = x, minY = y;
                }
                swap(current[x][y], current[row_0][col_0]); // 复原0和相邻位置
            }
        }
        if (curManha > minMan) { // 最小值优于当前状态
            swap(current[row_0][col_0], current[minX][minY]);
            row_0 = minX, col_0 = minY;
        }
        if (check()) { // 成功找到解
            totalTrial += trial;
            return true;
        }
        return false;
    }
}
```

```
// 随机重新开始爬山法
int randomRestart() {
    bool find = false;
    while (!find) {
        initial();
        find = hillClimbing();
    }
    return find;
}
```

结果:
随机重启爬山法结果如下
问题解决率 1/1000
搜索耗散 13

CSDN @MatsumotoChirikk

② 八皇后问题

```
// 爬山法
bool hillClimbing() {
    // 尝试次数大于100则判定为无解
    int i, j, k;
    for (int trial = 0; trial <= 100; trial++) {
        // 将当前初始状态数据复制到temp
        for (i = 0; i < 8; i++) {
            for (j = 0; j < 8; j++) {
                temp[i][j] = queens[i][j];
            }
        }
        int h[8][8];
        int minH = 9999, minX = 0, minY = 0, curState;
        for (i = 0; i < 8; i++) {
            for (j = 0; j < 8; j++) {
                for (k = 0; k < 8; k++) {
                    temp[i][k] = 0; // 在计算h(i, j)之前, 对i行所有位置赋值都为0
                    h[i][j] = findCollision(i, j); // 查找h(i, j)
                    if (queens[i][j] == 1) {
                        curState = h[i][j]; // 当前状态的值
                    }
                    // 先找出冲突个数最小的位置
                    if (h[i][j] < minH) {
                        minH = h[i][j];
                        minX = i;
                        minY = j;
                    }
                    for (k = 0; k < 8; k++) {
                        temp[i][k] = queens[i][k]; // 计算h(i, j)之后恢复原数据, 避免计算错误
                    }
                }
            }
        }
        // 将皇后放在该行冲突最少的位置处
        if (curState > minH) {
            for (int i = 0; i < 8; i++) {
                queens[minX][i] = 0;
                queens[i][minX] = 1;
            }
            // 判断是否找到解, 有解则返回值为真
            if (check(h)) {
                totalTrial += trial;
                return true;
            }
        }
        return false;
    }
}
```

```
// 随机重新开始爬山法
int randomRestart() {
    bool find = false;
    while (!find) {
        initial();
        find = hillClimbing();
    }
    return find;
}
```

随机重启爬山爬结果如下
问题解决率: 1/1000
搜索耗散: 5

CSDN @MatsumotoChirikk

① 八数码问题

```
// 模拟退火算法
bool simulated() {
    double temperature = 5; // 初始温度
    int trial = 0;
    while (temperature > 0.00001) {
        vector<int> v; // 记录可行的方向
        for (int i = 0; i < 4; i++) {
            int x = row_0 + direction[i][0];
            int y = col_0 + direction[i][1];
            if (x >= 0 && x < 3 && y >= 0 && y < 3) { // 方向可行
                v.push_back(i);
            }
        }
        int curManha = Manhattan(); // 当前状态的曼哈顿距离之和
        int dir = v[rand() % v.size()]; // 随机选取一个可行方向
        int x = row_0 + direction[dir][0];
        int y = col_0 + direction[dir][1];
        swap(currentrow_0[col_0], current[x][y]); // 交换0和相邻节点的位置
        int nextManha = Manhattan(); // 交换之后的曼哈顿距离之和
        int E = nextManha - curManha;
        if (E < 0) { // 下一状态优于当前状态
            row_0 = x, col_0 = y; // 更新0的位置
            trial++;
        } else if (exp((-1)*E/temperature) > ((double)rand() % 1000) / 10) {
            row_0 = x, col_0 = y; // Possible n
            trial++;
        } else { // 不成功的话，置0和相邻节点的位置
            swap(current[x][y], current[row_0][col_0]);
        }
        temperature -= 0.999; // 温度下降
        if (check()) { // 成功找到解
            totalTrial += trial;
            return true;
        }
    }
    return false;
}
```

结果:

模拟退火算法结果如下

问题解决率: 321/1000

搜索耗散: 1128

② 八皇后问题

```
// 模拟退火搜索
bool simulated() {
    double temperature = 5;
    int i, j, k;
    int trial = 0;
    while (temperature > 0.00001) {
        // 拷贝原始棋盘数据到temp
        for (i = 0; i < 8; i++) {
            for (j = 0; j < 8; j++) {
                temp[i][j] = queens[i][j];
            }
        }
        int h[8][8], curState;
        for (i = 0; i < 8; i++) {
            for (j = 0; j < 8; j++) {
                // 在计算h(i, j)之前，对i行所有位置赋值为0
                for (k = 0; k < 8; k++)
                    temp[i][k] = 0;
                // 查找h(i, j)
                h[i][j] = findCollision(i, j);
                // 当前状态的h值
                if (queens[i][j] == 1) {
                    curState = h[i][j];
                }
                // 计算h(i, j)之后要重置数据，避免计算错误
                for (k = 0; k < 8; k++)
                    temp[i][k] = queens[i][k];
            }
        }
    }
}
```

```
// 随机选取一个下一状态
bool better = false;
int next, nextState, times = 0;

next = rand() % 64;
nextState = h[next/8][next%8];
int E = nextState - curState;
if (E < 0) {
    better = true;
} else if (exp((-1)*E/temperature) > ((double)rand() % 1000) / 1000) {
    better = true;
}

if (better) {
    for (int i = 0; i < 8; i++)
        queens[next/8][i] = 0;
    queens[next/8][next%8] = 1; // 放置皇后
    trial++;
}

// 判断是否找到解，将解回溯置为真
if (check(h)) {
    totalTrial += trial;
    return true;
}

temperature -= 0.99;
return false;
}
```

模拟退火算法结果如下

问题解决率: 783/1000

CSDN @MatsumotoChirikk